



OPERACIONES BÁSICAS CON HIBERNATE

Guía: Operaciones Básicas con Hibernate aplicando el patrón DAO

Introducción

En esta lección aprenderemos a realizar operaciones básicas con Hibernate utilizando el patrón DAO (Data Access Object).

Aplicaremos la operación de **lectura (SELECT)** encapsulada dentro de una clase DAO.

Temas principales:

- Copia del proyecto base.
- Cambio en el archivo `persistence.xml`.
- Creación de la clase `PersonaDAO`.
- Implementación del método `listar()`.
- Prueba mediante la clase `OperacionesHibernateJPA`.



Paso 1: Modificar el archivo `persistence.xml`



Ruta del archivo:

HibernateEjemplo1/src/main/resources/META-INF/persistence.xml



Cambios realizados:

Se modificó el nombre de la unidad de persistencia:

```
<persistence-unit name="HibernatePU" transaction-type="RESOURCE_LOCAL">
```



Código completo `persistence.xml`:

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="3.0" xmlns="https://jakarta.ee/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
        https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd">

    <persistence-unit name="HibernatePU" transaction-type="RESOURCE_LOCAL">
        <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
        <class>gm.domain.Persona</class>
        <properties>
            <property name="jakarta.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/test?useSSL=true&useTimezone=true&serverTimezone=UTC"/>
            <property name="jakarta.persistence.jdbc.user" value="root"/>
            <property name="jakarta.persistence.jdbc.password" value="admin"/>
            <property name="jakarta.persistence.jdbc.driver" value="com.mysql.cj.jdbc.Driver"/>
        </properties>
    </persistence-unit>
</persistence>
```



Paso 2: Crear la clase DAO `PersonaDAO.java`



Ruta del archivo:

HibernateEjemplo1/src/main/java/gm/dao/PersonaDAO.java

Código:

```
package gm.dao;
```

```
import gm.domain.Persona;
import jakarta.persistence.*;

import java.util.List;

public class PersonaDAO implements AutoCloseable {

    private final EntityManagerFactory emf;
    private final EntityManager em;

    public PersonaDAO() {
        this.emf = Persistence.createEntityManagerFactory("HibernatePU");
        this.em = emf.createEntityManager();
    }

    public List<Persona> listar() {
        String hql = "SELECT p FROM Persona p";
        return em.createQuery(hql, Persona.class).getResultList();
    }

    public void imprimirPersonas() {
        listar().forEach(p -> System.out.println(" " + p));
    }

    @Override
    public void close() {
        if (em.isOpen()) em.close();
        if (emf.isOpen()) emf.close();
    }
}
```

Explicación paso a paso:

- **implements AutoCloseable**
Permite que esta clase pueda usarse en un bloque `try-with-resources`, para cerrar automáticamente las conexiones al finalizar el uso del DAO.
- **Atributos emf y em**
Se declaran como `final` y se inicializan en el constructor:
 - `emf`: Fábrica de gestores de entidades, creada usando la unidad de persistencia "HibernatePU".
 - `em`: Gestor de entidades, encargado de interactuar con la base de datos.
- **listar()**
Ejecuta una consulta HQL (`SELECT p FROM Persona p`) para recuperar todas las entidades `Persona` almacenadas.
Devuelve una `List<Persona>` con los resultados.

- `imprimirPersonas()`
Llama al método `listar()` y utiliza `forEach` con una expresión lambda para imprimir cada objeto `Persona` en consola.
- `close()` (sobrescrito del `AutoCloseable`)
Se encarga de cerrar correctamente el `EntityManager` y `EntityManagerFactory` si siguen abiertos. Esto es esencial para liberar recursos y evitar errores de conexión persistente en la aplicación.

✅ Con esta estructura, el acceso a datos se vuelve más limpio, eficiente y fácil de mantener.



Paso 3: Probar con `OperacionesHibernateJPA.java`



Ruta del archivo:

`HibernateEjemplo1/src/main/java/test/OperacionesHibernateJPA.java`

Código:

```
package test;

import gm.dao.PersonaDAO;

public class OperacionesHibernateJPA {
    public static void main(String[] args) {
        try (PersonaDAO personaDao = new PersonaDAO()) {
            // 1. Caso listar
            personaDao.imprimirPersonas();
        }
    }
}
```



Explicación del código:

- Se importa la clase `PersonaDAO`, que contiene la lógica para acceder a los datos de personas en la base de datos.
- Dentro del método `main`, se utiliza un **bloque try-with-resources** para crear una instancia de `PersonaDAO`.
Esto es posible porque `PersonaDAO` ahora implementa la interfaz `AutoCloseable`, lo que permite que sus recursos (como `EntityManager`) se cierren automáticamente al finalizar el bloque.
- Se llama al método `imprimirPersonas()` de `PersonaDAO`, que ejecuta una consulta para recuperar las personas almacenadas y las imprime en la consola.

✅ Esta versión mejora la gestión de recursos, evitando posibles fugas de conexión a la base de datos.

Paso 5: Ejecutar el proyecto

 Haz clic derecho sobre `OperacionesHibernateJPA.java` → **Run File**

Si todo está bien configurado, en la consola verás una salida similar a:

```
Hibernate: select p1_0.id_persona,p1_0.apellido,p1_0.email,p1_0.nombre,p1_0.telefono from persona
p1_0
Persona{idPersona=1, nombre=Rafael, apellido=Juarez, email=rjuarez@mail.com, telefono=55667788}
Persona{idPersona=2, nombre=Ivonne, apellido=Lara, email=ilara@mail.com, telefono=55443322}
Persona{idPersona=3, nombre=Maria, apellido=Esparza, email=mesparza@mail.com, telefono=55112233}
Persona{idPersona=5, nombre=Hugo, apellido=Hernandez, email=hhernandez@mail.com,
telefono=55778822}
```

Conclusión

En esta lección aplicamos correctamente el patrón **DAO (Data Access Object)** junto con Hibernate y JPA para realizar operaciones básicas sobre la entidad `Persona`.

◆ ¿Qué logramos?

- Encapsular la lógica de acceso a datos dentro de la clase `PersonaDAO`.
- Utilizar HQL (Hibernate Query Language) para recuperar objetos `Persona` desde la base de datos.
- Implementar la interfaz `AutoCloseable` para manejar recursos de forma automática.
- Usar un **bloque try-with-resources** para asegurarnos de que se cierren correctamente las conexiones (`EntityManager` y `EntityManagerFactory`) sin tener que hacerlo manualmente.
- Organizar y simplificar nuestro código, haciendo que sea más profesional, limpio y fácil de mantener.

 Este enfoque sienta una base sólida para las siguientes operaciones CRUD (crear, modificar y eliminar) que veremos más adelante en el curso, manteniendo una arquitectura robusta desde el inicio.

Sigue adelante con tu aprendizaje  , ¡el esfuerzo vale la pena!

¡Saludos! 

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)